Name: P. Lakshmi Gayathri Navaneetha
REG NO: 192321169

CSA17 – ARTIFICIAL INTELLIGENCE

Assignment 1 – CO1: Design, Analyse and Validate an Efficient Search Strategy

Hospital Medicine-Delivery Robot | Tasks 1–12 (50 marks)

Summary of findings

- **Strategies compared:** BFS (iterative, queue), Uniform-Cost Search (iterative, priority queue) and Iterative-Deepening DFS (recursive depth-limited search, repeated). All three return correct routes on generated graphs of 1,000 / 10,000 / 100,000 / 1,000,000 states (seed 17).
- **Measured at 1,000,000 states:** BFS generated 853,312 nodes in 1.58 s; UCS 976,422 nodes in 6.21 s; IDDFS 6,877,254 nodes in 6.12 s but with a frontier of only 13 nodes (BFS: 186,223).
- **Recommendation:** UCS on adjacency lists (run once from the pharmacy, which is always the start). It is the only strategy that minimises travel time; BFS was up to 13% slower in travel time on the test routes.
- **Day-4 change:** the original design cannot be reused as-is. Searching belief states without a sensor exploded (cap of 150,000 beliefs reached in 9/10 trials at k = 4 and 10/10 at k = 8), while adding a local wall sensor solved every trial with about 53 belief states at k = 8. Decision: MODIFY (belief-state/conditional search for localisation, then reuse UCS for navigation).

Honesty notes. The hospital graphs are synthetic (random corridor tree plus 10% cross-links). Timings are for pure Python on one machine. IDDFS has a node budget so that it terminates; it hit that budget on every unreachable-goal test. The belief-state experiment uses a small 12×12 floor plan (124 open cells), 10 random trials per k, with search caps stated where they were hit.

Task 1 – Design the Agent and the Search Algorithms (5 marks)

1.1 PEAS description

Component	Description
Performance measure	Correct medicine delivered to the correct ward; minimum total travel time; no collisions or spills; deliveries on time (priority orders first); low battery use.
Environment	Multi-block hospital modelled as a graph: junctions, corridors, lifts, ward doors, pharmacy docking station. People, trolleys and other robots are present but treated as obstacles.
Actuators	Drive motors/wheels, lift-button arm, drawer lock/release for the medicine box, door opener, buzzer/display.
Sensors	Localisation sensor (beacons/RFID/LiDAR), wheel odometry, bump and obstacle sensors, camera, ward-door ID reader, battery monitor.

1.2 Environment classification

Property	Choice	Justification
Observability	Fully observable (original design)	The localisation sensor gives the exact junction and the map is known. On Day 4 this becomes partially observable.
Determinism	Deterministic (planning model)	Moving between two connected junctions has one outcome. Rare real-world slips are handled by re-planning, not modelled.
Episodic / sequential	Sequential	Each move changes the position and so the cost of every later move; a route is a chain of dependent decisions.
Static / dynamic	Static during planning	The map and travel times do not change while the plan is computed. Closed corridors are treated as a re-planning event.
Discrete / continuous	Discrete	Finite set of junctions, finite set of moves, integer travel times.

Property	Choice	Justification
Single / multi-agent	Single-agent	One robot plans; other moving objects are obstacles, not competitors or cooperators.

1.3 View of AI and rationality

The robot follows the **acting rationally** view: it is a rational agent that picks the action sequence expected to maximise its performance measure (least travel time to the right ward) given its percepts and knowledge. It is not trying to imitate humans (acting/thinking humanly), and although logic helps to formulate the problem, correct behaviour rather than correct reasoning is the aim (so not thinking rationally).

It is a **goal-based problem-solving agent**: it formulates a goal (be at ward w), formulates the problem (below), searches for a sequence of actions, then executes it. Because the environment is observable, deterministic and known, the solution is a fixed sequence that can be executed without further percepts.

Historical link. Casting tasks as search through a state space goes back to the Logic Theorist (1956) and the General Problem Solver (Newell and Simon, late 1950s), and to route-planning robots such as Shakey (1960s), whose planner led to A*. Route-finding for a delivery robot is a direct descendant of that problem-solving tradition.

1.4 Problem formulation

Element	Definition
States	A location v in V (junction, corridor point, lift, ward door); |V| grows 10^3 → 10^6+.
Initial state	The pharmacy docking station, node 0.
Actions	ACTIONS(v) = { Go(u) : (v,u) is an edge }, i.e. move to a connected location.
Transition model	RESULT(v, Go(u)) = u.
Goal test	v = requested ward w.
Path cost	Sum of travel times c(v,u) of the moves; each c ≥ ε = 1 minute (integers 1–10 in the experiments).

1.5 The three search strategies

	A. BFS	B. Uniform-Cost Search	C. IDDFS
Style	Iterative, FIFO queue	Iterative, priority queue (min-heap)	Recursive depth-limited DFS, repeated
Idea	Expand the shallowest node first, level by level.	Expand the node with the lowest path cost g(n) first.	Run DFS with limit L = 0, 1, 2, …; keeps only the current path in memory.
Why chosen	Baseline; finds the route with fewest moves; cheap per node.	Travel times differ (lifts, long corridors), so the least-time route needs cost-ordered search.	Memory is the worry as the hospital grows; also gives the recursive algorithm needed for Tasks 6–7.
Preprocessing	Edge list → adjacency lists; reached/parent map.	Adjacency lists with weights; best-cost map; closed set.	Adjacency lists only; an on-path set for cycle checks. No explored set.

Preprocessing (common to all)

```
BUILD-ADJACENCY(V, edge_list):          # Theta(V + E) time and space
  Adj[v] <- empty list for every v in V
  for each (u, v, w) in edge_list:
      Adj[u].append((v, w));  Adj[v].append((u, w))
  return Adj
```

Pseudocode A – BFS (iterative)

```
BFS(s, g):
  if s == g: return [s]
  reached <- {s: None};  frontier <- queue([s])
  while frontier not empty:
      u <- frontier.pop_front()                # node expansion
      for v in Adj[u]:                         # successor generation
          if v in reached: continue            # duplicate detection
          reached[v] <- u
          if v == g: return PATH(reached, v)   # goal test at generation
          frontier.push_back(v)
  return FAILURE
```

Pseudocode B – Uniform-Cost Search (iterative)

```
UCS(s, g):
  frontier <- min-heap{(0, s)};  best <- {s: 0};  parent <- {s: None};  done <- {}
  while frontier not empty:
      (gc, u) <- frontier.pop_min()            # node expansion, O(log n)
      if u in done: continue                   # stale heap entry
      if u == g: return PATH(parent, u)        # goal test at expansion => optimal
      done.add(u)
      for (v, w) in Adj[u]:                    # successor generation
          if v not in done and gc + w < best.get(v, infinity):
              best[v] <- gc + w;  parent[v] <- u
              frontier.push((gc + w, v))       # frontier insertion, O(log n)
  return FAILURE
```

Pseudocode C – IDDFS (recursive)

```
IDDFS(s, g):
  for L = 0, 1, 2, ...:
      path <- [s]
      r <- DLS(s, g, L, path)
      if r == FOUND:  return path
      if r == FAIL:   return FAILURE           # tree exhausted without any cutoff

DLS(u, g, L, path):                            # recursive
  if u == g: return FOUND
  if L == 0: return CUTOFF
  cutoff <- false
  for v in Adj[u]:
      if v in path: continue                   # avoid cycles on the current path only
      path.push(v)
      r <- DLS(v, g, L - 1, path)              # recursive call
      if r == FOUND: return FOUND
      path.pop()
      if r == CUTOFF: cutoff <- true
  return CUTOFF if cutoff else FAIL
```

The three are meaningfully different: they use different frontier structures (queue, heap, call stack), have different guarantees (fewest moves, least time, fewest moves with O(d) memory) and different duplicate handling (explored set, closed set with best-cost map, path-only check).

Task 2 – Identify the Basic Operation (2 marks)

Algorithm	Basic operation	Where it occurs	Why it is appropriate / how its count changes
BFS	Successor generation with duplicate test: generating v from u and checking v in reached.	Inner for-loop of the expansion step.	It is the innermost, most frequently executed step; pop, append and dictionary insert happen a constant number of times per generation, so running time is proportional to its count. Count: b + b^2 + … + b^d in a tree; in a graph at most Σdeg(u) = 2E, since each node is expanded once. As the hospital grows from 10^3 to 10^6 states the count grows roughly linearly with V.
UCS	Priority-queue operations (heap push at frontier insertion, heap pop at	pop_min at the top of the loop; push inside the inner loop.	Each generated improving node costs one push and each expanded node one pop, so time ≈ (generated + expanded) × log(frontier). The count grows like BFS

Algorithm	Basic operation	Where it occurs	Why it is appropriate / how its count changes
	expansion), each costing O(log n), driven by successor generation.		but the log factor and the extra levels explored (all nodes with g ≤ C*) make it grow faster: depth ⌊C*/ε⌋ + 1 instead of d.
IDDFS	The recursive call DLS(v, …): one visit = successor scan + cycle check + goal test.	The recursive call inside the for-loop of DLS.	Every visited node costs a constant amount, and iterations repeat the upper levels, so time is proportional to the total number of calls. Level i is visited (d − i + 1) times. As V grows d grows (about +3 levels per ×10 states here) and the count multiplies by b for every extra level, so it grows faster than V.

Task 3 – Derive Time and Space Complexity (6 marks)

3.1 Input size and model

Parameters: b = branching factor (successors per node other than the parent), d = depth of the shallowest goal, m = maximum depth, C* = optimal cost, ε = minimum step cost, V = number of states, E = number of edges. In the tree model every node has b children, so level i has b^i nodes. In a graph with duplicate detection no node is expanded twice, so every count is also capped by V (or by E for edge scans).

3.2 Preprocessing

Building adjacency lists processes each edge once and appends it twice, so P(V,E) = Θ(V + E) time and Θ(V + E) space. For the hospital graphs E ≈ 1.1V, so P = Θ(V). BFS and UCS also keep a reached/closed map, which grows lazily up to Θ(V). IDDFS needs no explored set, only an on-path set of size ≤ d.

3.3 BFS

With the goal test at generation, the worst case generates every node on levels 1 to d:

```
T_BFS(d) = b + b² + … + b^d = Σ_{i=1..d} b^i = (b^(d+1) − b)/(b − 1) = Θ(b^d)
Check: b = 2, d = 3 → 2 + 4 + 8 = 14 = (16 − 2)/1.
```

Memory: all generated nodes are kept (frontier plus reached set), so S_BFS = Σ b^i = Θ(b^d). The frontier alone holds the last level, up to b^d nodes.

In a graph each node is expanded once and each adjacency entry scanned once, so T_BFS ≤ V expansions and ≤ 2E generation checks: Θ(V + E) in the worst case, S = Θ(V).

Overall (preprocess + search): Θ(V + E) + O(min(b^d, V + E)) = Θ(V + E). The search can never cost more than the preprocessing in order of growth, so for one delivery the adjacency build is as expensive as the search; over many deliveries it is paid once.

3.4 Uniform-Cost Search

UCS expands every node with g(n) < C* (and some with g = C*). Each step costs at least ε, so no such node is deeper than ⌊C*/ε⌋. Writing D = 1 + ⌊C*/ε⌋ for the effective depth:

```
N_UCS = Σ_{i=0..D} b^i = (b^(D+1) − 1)/(b − 1) = Θ(b^D) = Θ(b^(1+⌊C*/ε⌋))
T_UCS = O(N_UCS · log N_UCS) because each node needs a heap push and a pop of O(log |frontier|)
S_UCS = Θ(b^(1+⌊C*/ε⌋))
```

When every step costs ε, C*/ε = d and UCS explores one level deeper than BFS (b^(d+1)) because it tests the goal at expansion. In a graph with a closed set and a binary heap: T = O((V + E) log V), S = Θ(V).

Overall: Θ(V + E) preprocessing + O((V + E) log V) search = O((V + E) log V).

3.5 IDDFS (with recursive DLS)

Iteration with limit L generates levels 1..L once each, so its cost is Σ_{i=1..L} b^i. Level i (i ≤ d) is therefore generated in iterations L = i, …, d, which is (d − i + 1) times:

```
T_IDDFS(d) = Σ_{i=1..d} (d − i + 1)·b^i = d·b + (d − 1)·b² + … + 1·b^d
Closed form: Σ_{i=0..d} (d + 1 − i) b^i = (b^(d+2) − (d + 2)b + (d + 1))/(b − 1)²,
so
T_IDDFS(d) = (b^(d+2) − (d + 2)b + (d + 1))/(b − 1)² − (d + 1) ≈ b^(d+2)/(b − 1)² =
Θ(b^d)
Check: d = 2 → b² + 2b (level 1 twice, level 2 once); formula gives (b⁴ − 4b + 3)/(b
− 1)² − 3 = b² + 2b ✓.
```

Compared with a BFS-like count b^(d+1)/(b − 1), the repeated work multiplies the cost by only b/(b − 1) (2 when b = 2, 1.11 when b = 10). Memory is the current path plus the loop state: S_IDDFS = Θ(d) with lazy successor iteration (O(b·d) if all successors are stored).

Overall: Θ(V + E) + Θ(b^d). Without an explored set, IDDFS counts paths, not nodes: on a graph with cycles b^d can exceed V + E by a large margin, so the search term dominates (observed in Task 9–10).

Task 4 – Best, Average and Worst Cases (4 marks)

4.1 Conditions and reasoning

Algorithm	Best case	Average case	Worst case
BFS	Start = goal (0 generated) or goal is the first successor of the start (1 generated, early goal test): Θ(1).	Goal uniformly placed in its level: all shallower levels plus about half of level d, Σ_{i<d} b^i + (b^d + 1)/2 = Θ(b^d). With random wards the depth is typically below the maximum, so the count is well below V.	Goal last in the deepest level, or unreachable: the whole connected part is explored, Θ(V + E). Termination is guaranteed by the explored set.
UCS	Start = goal, or goal is the cheapest neighbour: only nodes cheaper than the goal are expanded, Θ(b).	All nodes with g below the cost of a typical ward are expanded; this set is larger than the BFS set because cheap edges let the frontier go deeper.	Cheap-edge detours (many ε-cost paths that stay below C*) push the effective depth to ⌊C*/ε⌋ + 1 ≫ d, or the ward is unreachable: Θ((V + E) log V).
IDDFS	Start = goal: 0. For a goal at depth d on the first-explored branch the final pass costs only about d nodes, but the earlier passes must still exhaust their trees: Σ_{L<d} b^L ≈ b^(d−1).	The last pass explores about half of the depth-d tree: ≈ ½ b^d, plus the earlier passes (factor b/(b − 1) in total).	Goal last in the final pass: Σ (d − i + 1) b^i. Unreachable: IDDFS enumerates every simple path of the component, exponential in V on a cyclic graph, so it does not terminate in practice.

4.2 Experimental support (V = 10,000, nodes generated)

Case	BFS	UCS	IDDFS
Best: start = goal	0	0	0
Best: goal = first neighbour	1	39	1
Average: 30 random goals (mean depth 7.4)	5,127	5,984	25,888
Worst: deepest ward (depth 15)	9,995	10,283	2,040,864
Worst: unreachable ward	9,995	10,283	>3,000,000 (budget)

Best cases cost 0–39 nodes (the first neighbour of the pharmacy was not its cheapest one, so UCS generated 39 rather than Θ(b)). The average over 30 random wards (mean depth 7.4) is roughly 50–60% of V for BFS and UCS. The deepest ward (depth 15) and an unreachable ward force BFS and UCS to explore the whole component (9,996 nodes), whereas IDDFS needed 2,040,864 generations for the deepest ward and did not finish for the unreachable one within 3,000,000.

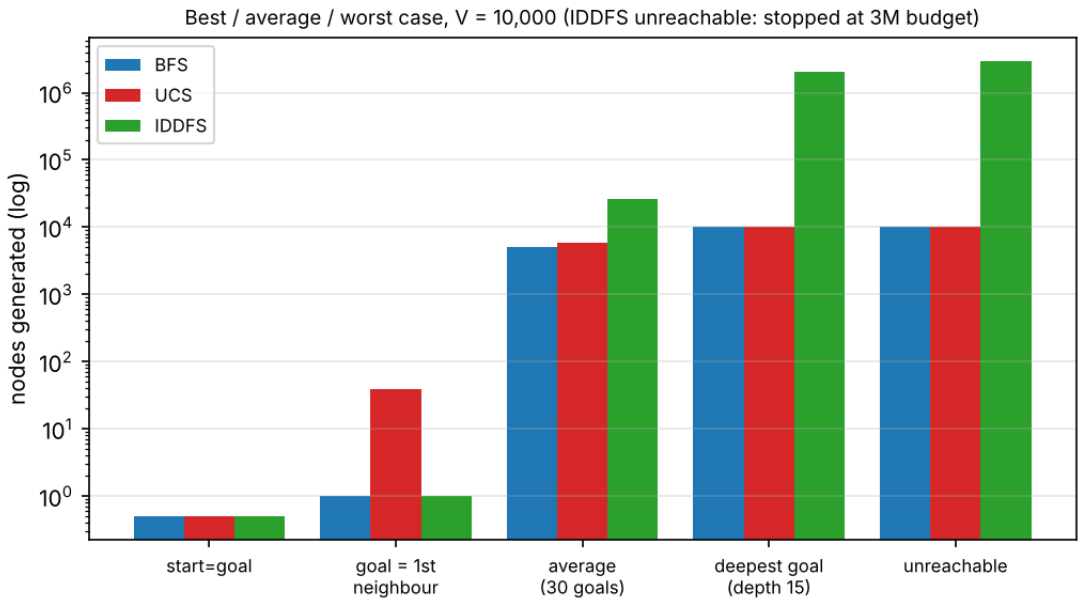


Figure 1 – Nodes generated in best/average/worst cases (log scale).

Task 5 – Asymptotic Analysis (3 marks)

Algorithm	Time (graph, worst)	Time (tree model)	Memory	Best-case time
BFS	Θ(V + E) = Θ(V)	Θ(b^d)	Θ(V) (frontier Θ(b^d))	Ω(1)
UCS	Θ((V + E) log V)	O(b^(1+⌊C*/ε⌋))	Θ(V)	Ω(1)
IDDFS	Θ(b^d), can exceed V + E	Θ(b^d)	Θ(d)	Ω(b^(d−1)) for a goal at depth d; Ω(1) for start = goal

Large state spaces. In the generated hospital graphs E ≈ 1.1V and the maximum depth grew only logarithmically (12, 15, 18, 21 for ×10 steps in V). BFS and UCS therefore scale linearly in V (UCS with a log factor) and their memory also grows linearly: the frontier was about 19% of V at every size. IDDFS memory is Θ(d) = Θ(log V), 10 to 13 nodes in the experiments, but its time grows as b^d with d increasing by about 3 per tenfold growth, so it grows faster than V (nodes generated grew ×4, ×8 and ×12 per step in the experiments).

Meaning for the hospital robot. Going from 1,000 to 1,000,000 states, BFS and UCS keep answering in seconds but their memory grows by 1,000×: at 10^6 states the frontier reached 186,000–199,000 entries plus a 0.85–0.98 million-entry reached/closed map, so **memory is the first limit** for BFS/UCS in a real, much larger map. IDDFS never runs out of memory, but its **time is the first limit**: it needed 6.9 million node generations at 10^6 states, about 8× BFS, and the gap widens as depth grows. Because the pharmacy is always the start, the extra preprocessing cost is paid once and shared by every delivery.

Task 6 – Recurrence Relation for the Recursive Algorithm (3 marks)

The recursive algorithm is the depth-limited search DLS(u, g, L, path) inside IDDFS. Let T(L) be the number of operations (recursive calls) made by one call DLS(u, ·, L, ·) in the worst case, i.e. when the goal is not found and every successor is explored.

Component	Value (derived from the algorithm)
Size of the original problem	n = L, the remaining depth limit at the current node.
Size of each subproblem	L − 1: each call passes limit L − 1 to the child (a decrease by one, not a division).
Number of recursive calls	b: the for-loop calls DLS once for each of the b successors that are not already on the path.

Component	Value (derived from the algorithm)
Work outside the recursive calls	c: goal test, limit test, loop control and the on-path membership checks of the b successors, a constant independent of L.
Base case	L = 0: the call does the goal test and returns CUTOFF, no children: T(0) = c.

Putting these together:

$$T(L) = b \cdot T(L-1) + c \quad \text{for } L \geq 1, \qquad T(0) = c$$

For the whole IDDFS with final limit d, the work is the sum over iterations:

$$I(d) = \Sigma_{L=0..d}\, T(L)$$

Task 7 – Solve the Recurrence (4 marks)

Method. The recurrence is a linear recurrence that reduces the size by one (T(L) = a·T(L − 1) + c). The Master theorem handles divide-and-conquer recurrences T(n) = aT(n/b) + f(n) and does not apply here, so I use **iteration (backward substitution)**, then confirm with a recursion tree and an induction check.

Steps

$$T(L) = b \cdot T(L-1) + c$$
$$= b \cdot (b \cdot T(L-2) + c) + c = b^2 \cdot T(L-2) + c(b+1)$$
$$= b^3 \cdot T(L-3) + c(b^2 + b + 1)$$
$$\ldots \quad \text{after } k \text{ substitutions:} \quad T(L) = b^k \cdot T(L-k) + c(b^{k-1} + \ldots + b + 1) = b^k \cdot T(L-k) + c(b^k - 1)/(b-1)$$

Set $k = L$ and use $T(0) = c$:

$$T(L) = b^L \cdot c + c(b^L - 1)/(b-1) = c(b^{L+1} - 1)/(b-1)$$

Checks

- **Recursion tree:** level j has b^j calls, each costing c, for j = 0..L, so the total is c·Σ b^j = c(b^(L+1) − 1)/(b − 1), the same result.
- **Induction:** the formula gives T(0) = c. If T(L − 1) = c(b^L − 1)/(b − 1), then b·T(L − 1) + c = c(b^(L+1) − b + b − 1)/(b − 1) = c(b^(L+1) − 1)/(b − 1). ✓

Result and meaning

$$T(L) = c(b^{L+1} - 1)/(b-1) = \Theta(b^L) \quad (\text{for } b \geq 2); \quad \text{for } b = 1 \text{ (a corridor) } T(L) = c(L+1) = \Theta(L)$$

Each extra level of depth multiplies the work by b, so the cost is exponential in depth and only polynomial in b for fixed depth. This matches the experiment: the nodes generated per depth limit grew by a factor of about 2.05 per level (Task 10), i.e. b ≈ 2.05.

Extension to iterative deepening

$$I(d) = \Sigma_{L=0..d}\, c(b^{L+1} - 1)/(b-1) = c/(b-1) \cdot [\, b(b^{d+1} - 1)/(b-1) - (d+1) \,]$$
$$= c[\, b^{d+2} - b - (d+1)(b-1) \,]/(b-1)^2 = \Theta(b^d)$$
$$I(d)/T(d) \to [b^{d+2}/(b-1)^2] \,/\, [b^{d+1}/(b-1)] = b/(b-1)$$

Repeating the search for increasing limits costs only a constant factor b/(b − 1) more than a single depth-d search, 1.95 for b = 2.05, so IDDFS keeps the O(b^d) time of BFS and uses O(d) memory. In the 100,000-state graph the cumulative nodes generated up to limit 17 (40.4 million) divided by the last pass (20.6 million) was 1.96.

Task 8 – Implement and Test (4 marks)

All three algorithms are implemented in Python 3.12 in hospital_search.py (full listing in the appendix). They are the same algorithms as in Tasks 1–7: BFS with a deque and reached map, UCS with heapq, and recursive IDDFS with an on-path set.

8.1 Test graphs and reproducibility

- **Generator:** random.Random(17). Main part: a random corridor tree (node i attaches to a random earlier node) plus 10% extra cross-links (bridges/lifts); travel times are random integers 1–10 minutes. The last 4 nodes form a separate sealed wing so that one ward is unreachable. Node 0 is the pharmacy.
- **Goal selection:** random.Random(17 + V) picks a ward at depth ≥ 60% of the maximum depth (same pair for all algorithms). **Belief experiment:** floor plan Random(17), trials Random(29).
- **Commands:** python run_experiments.py 1000 10000 100000; python run_experiments.py 1000000; python exp_cases.py; python belief_search.py 2 4 8; python make_figs.py.

V	Edges E	Avg degree	Max depth	Preprocessing (adjacency)	Graph generation
1,000	1,097	2.19	12	0.24 ms	2.12 ms
10,000	10,997	2.20	15	5.64 ms	11.3 ms
100,000	109,997	2.20	18	113.1 ms	132.8 ms
1,000,000	1,099,996	2.20	21	2.81 s	2.28 s

8.2 Test cases and outputs

Expected behaviour: for a reachable ward all algorithms return a valid route; UCS returns the minimum-time route; BFS and IDDFS return a route with the fewest moves; for start = goal the route is [start] with cost 0; for the unreachable ward BFS and UCS report failure.

V	Ward	Reachable ward (moves, minutes)	Start = goal	Unreachable ward
1,000	904 (depth 10)	BFS 10 moves, 60 min UCS 10 moves, 60 min IDDFS 10 moves, 60 min	all three: 0 moves, 0 min	BFS no route UCS no route IDDFS budget exceeded
10,000	9091 (depth 10)	BFS 10 moves, 54 min UCS 11 moves, 50 min IDDFS 10 moves, 54 min	all three: 0 moves, 0 min	BFS no route UCS no route IDDFS budget exceeded
100,000	30463 (depth 11)	BFS 11 moves, 58 min UCS 11 moves, 58 min IDDFS 11 moves, 58 min	all three: 0 moves, 0 min	BFS no route UCS no route IDDFS budget exceeded
1,000,000	665318 (depth 13)	BFS 13 moves, 80 min UCS 15 moves, 71 min IDDFS 13 moves, 80 min	all three: 0 moves, 0 min	BFS no route UCS no route IDDFS budget exceeded

Observations. All routes are valid and have the expected optimality: where BFS/IDDFS and UCS differ (10,000 and 1,000,000 states) the BFS route has fewer moves but costs more minutes, as theory predicts. The unreachable test confirms the Task 4 worst case: BFS and UCS finish by exhausting the component, while IDDFS stops only at its node budget (2,000,000 here) because without an explored set it keeps enumerating paths.

Task 9 – Experimental Performance (3 marks)

One fixed reachable start–goal pair per size, identical for all algorithms. Time is the mean over the runs shown. IDDFS 'frontier' is the maximum recursion/path depth (its memory proxy). 'Not min-cost' means a cheaper route exists (UCS cost).

V	Algorithm	Generated	Expanded	Max frontier	Depth / cost (min)	Time
1,000	BFS	987	968	215	10 / 60	0.22 ms (5 runs)
	UCS	1,019	987	238	10 / 60	0.76 ms (5 runs)
	IDDFS	17,500	9,691	10	10 / 60	4.76 ms (5 runs)
10,000	BFS	9,208	8,157	2,040	10 / 54 (not min-cost)	3.12 ms (5 runs)
	UCS	9,525	8,374	2,135	11 / 50	11.1 ms (5 runs)

V	Algorithm	Generated	Expanded	Max frontier	Depth / cost (min)	Time
	IDDFS	74,139	39,502	10	10 / 54 (not min-cost)	22.2 ms (5 runs)
100,000	BFS	84,960	70,395	19,082	11 / 58	66.8 ms (5 runs)
	UCS	92,132	77,657	20,025	11 / 58	228.9 ms (5 runs)
	IDDFS	572,971	297,933	11	11 / 58	367.3 ms (5 runs)
1,000,000	BFS	853,312	711,773	186,223	13 / 80 (not min-cost)	1.58 s (5 runs)
	UCS	976,422	876,141	199,327	15 / 71	6.21 s (5 runs)
	IDDFS	6,877,254	3,478,613	13	13 / 80 (not min-cost)	6.12 s (5 runs)

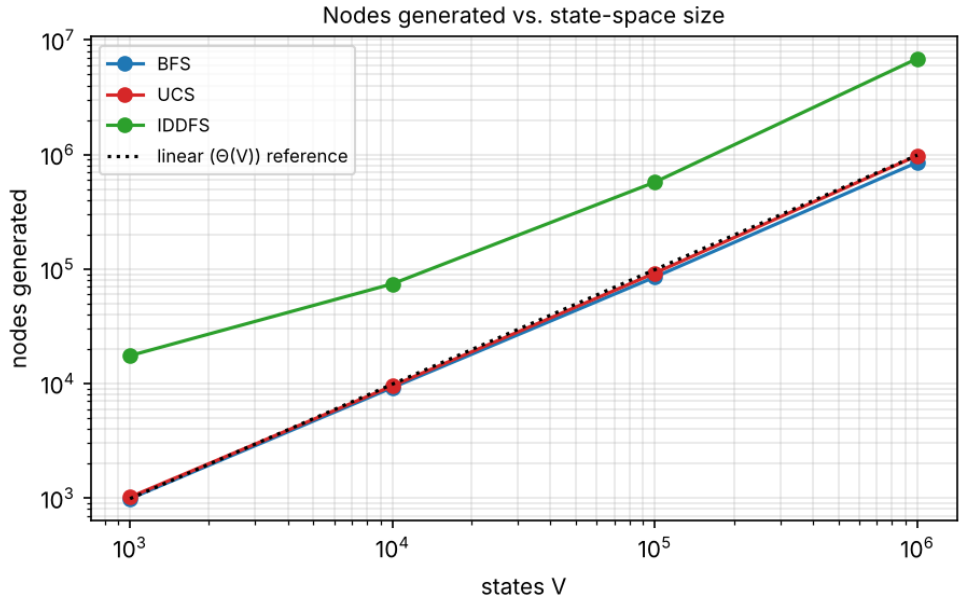


Figure 2 – Nodes generated vs. V (log–log). BFS/UCS follow the linear reference; IDDFS grows faster.

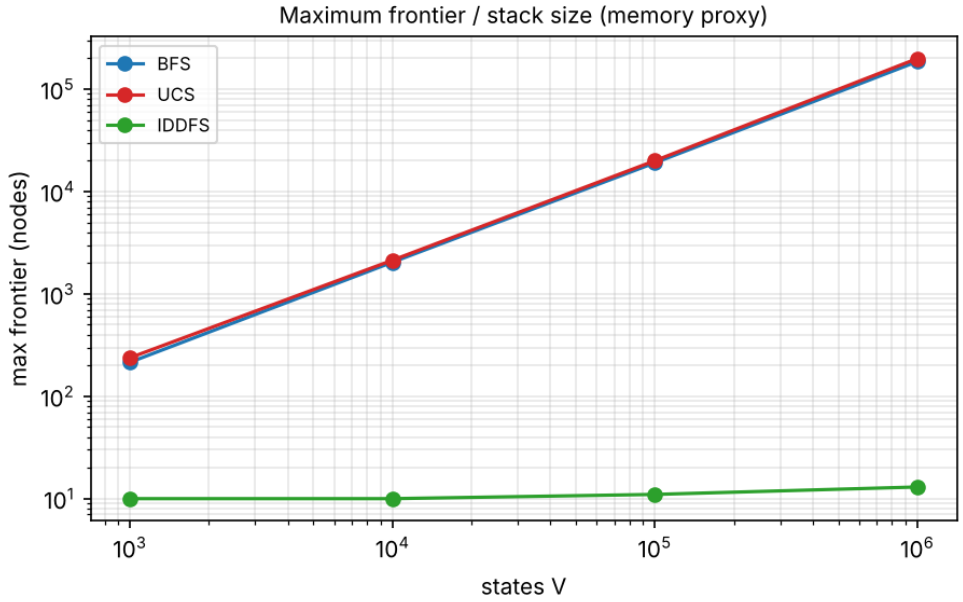


Figure 3 – Maximum frontier size (memory proxy). BFS and UCS are linear in V, IDDFS stays at 10–13 nodes.

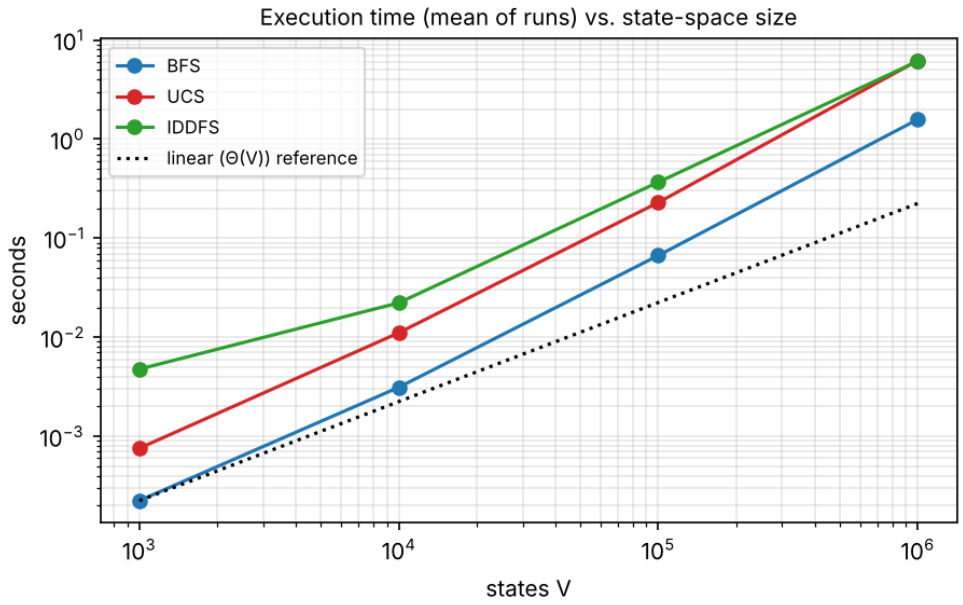


Figure 4 – Execution time vs. V (log–log).

Task 10 – Compare Theory with Experiment (4 marks)

10.1 Nodes generated

BFS generated 987 → 853,312 nodes over a 1,000× increase of V, a log–log slope of about 0.98, which agrees with the graph-search bound Θ(V + E) (and not with b^d from the tree model). The reason is the explored set: every node is generated at most once, so the count saturates at V. The fraction of V generated (0.99, 0.92, 0.85, 0.85) is below 1 only because the target ward was found before the last level was complete. Using the tree formula with an effective branching factor b_eff = generated^(1/d) gives 1.99, 2.49, 2.81, 2.86 for the four sizes. This b_eff is not constant, because this tree has high-degree hubs near the pharmacy and many leaves, so a single b is only a rough summary of the graph.

UCS generated 3–14% more nodes than BFS and expanded more (876,141 vs 711,773 at V = 10^6), as predicted: it must exhaust everything cheaper than the goal, which lies at depth 15 for its cheaper route compared with depth 13 for BFS's route.

IDDFS generated 17.7×, 8.1×, 6.7× and 8.1× as many nodes as BFS. The Task 3 repeated-work factor b/(b − 1) ≈ 2 explains only a small part of this. The main cause is **duplicates**: with no explored set the same junction is reached again by every alternative path, and the 10% cross-links create many alternatives, so IDDFS counts paths rather than nodes.

Depth limit L (V = 100,000)	Nodes generated in the pass	Ratio to previous pass
8	42,518	-
9	80,557	1.89
10	152,090	1.89
11	293,232	1.93
12	581,203	1.98
13	1,174,375	2.02
14	2,399,985	2.04
15	4,925,946	2.05
16	10,096,380	2.05
17	20,610,130	2.04

The per-pass ratio settles at about 2.05, giving b ≈ 2.05 as in Task 7. At L = 17 one pass generates 20.6 million nodes in a 100,000-node graph, 206 times V, so the graph really does contain exponentially many paths. The measured ratio between cumulative work and the last pass, 1.96, matches the predicted b/(b − 1) = 1.95.

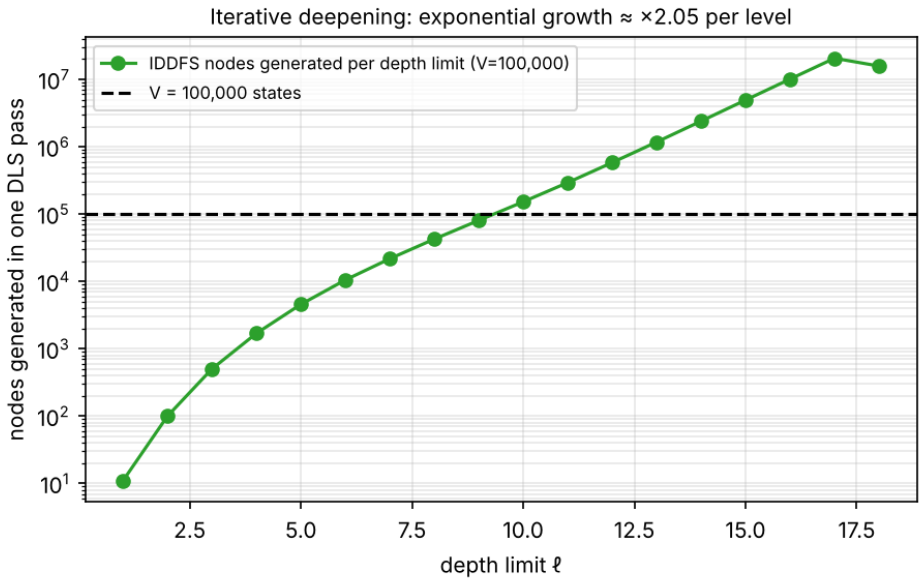


Figure 5 – Nodes generated by each IDDFS pass (V = 100,000).

10.2 Frontier size

BFS frontier divided by V was 0.215, 0.204, 0.191, 0.186, so the observed memory is Θ(V) as derived (about 0.19V, because the tree has many leaves). UCS is within 10% of that. The IDDFS stack depth of 10, 11 and 13 follows Θ(d) with d ≈ log V.

10.3 Time and differences from theory

- **Time vs. operations:** BFS operations grew with slope 0.98 but its time rose 7,050× (slope about 1.28) when V grew 1,000×. Likely reasons: larger hash tables and lists no longer fit in the CPU cache, memory allocation and garbage collection, and Python object overhead. These are implementation effects, not algorithmic ones.
- **UCS vs. BFS time:** 3.4–3.9× slower per query for about 10% more nodes, consistent with the log(frontier) heap cost and tuple allocation.
- **IDDFS time vs. nodes:** IDDFS handled 6.9 million nodes in about the same 6 s that UCS needed for 0.98 million, because recursive calls are cheap compared with heap operations, so its time ranking is better than its node-count ranking.
- **Tie-breaking and graph structure:** which successor is listed first decides which node of a level is generated first, so counts for one goal vary by a few percent; the random-recursive-tree structure (hubs, logarithmic depth) also makes b non-uniform.

10.4 Which scales better?

For time, BFS scales best (fastest at every size, Θ(V)), UCS is a constant factor slower with the same growth, and IDDFS is worst because its work grows with the number of paths. For memory, IDDFS scales best (Θ(log V)) and BFS/UCS are Θ(V). The experimental growth agrees with the theory in every case, with the exceptions that the tree-model b^d is replaced by Θ(V) for the explored-set algorithms and the IDDFS overhead is much larger than b/(b − 1) because of duplicate paths.

Task 11 – Recommend and Defend the Final Design (4 marks)

Recommendation: Uniform-Cost Search on adjacency lists, run from the pharmacy.

- **Why it is appropriate:** the robot's objective is minimum delivery time and travel times differ between moves. UCS is the only one of the three that guarantees the least-time route. BFS returned a route that was slower in 2 of 4 test cases (54 vs 50 minutes at 10,000 states, 80 vs 71 minutes at 1,000,000 states: 8% and 13% slower) even though it needed 3.4–3.9× less time to compute.
- **Why not the others:** BFS is only optimal if all moves cost the same, which they do not. IDDFS saves memory (13 nodes vs ~199,000) but needed about 6.2× the node generations of UCS at 100,000 states and 7.0× at 10^6, and it never terminates on an unreachable ward, which is an unacceptable failure mode for a real robot.
- **Efficiency:** theory $O((V + E) \log V)$ time and $\Theta(V)$ memory; measured 6.21 s per query at 10^6 states in Python and 228.9 ms at 10^5. Adjacency-list preprocessing, $\Theta(V + E)$, measured 2.81 s at 10^6 states.
- **Preprocessing and trade-offs:** the adjacency lists are built once and reused. Because every trip starts at the pharmacy, a single UCS run to exhaustion yields the least-time route to every ward; later requests then only read a stored parent chain. This is a design suggestion that follows from the algorithm; I did not time it.
- **Limitations and growth:** memory is linear in V (about 0.2V frontier plus the closed set), which is acceptable at 10^6 states but is the first limit at much larger sizes. A compiled implementation would be faster than my Python timings, but I have not measured that. Cost-aware informed search (A*) would be the next step, but it is outside this assignment.
- **Fallback:** if all moves are given equal cost (for example to count corridor hops), BFS is the cheaper choice at the same optimality.

Task 12 – Day-4 Design Challenge (8 marks)

A. Impact analysis (2 marks)

- **What changed:** after a restart the robot knows only that it is at one of k junctions. The initial state is no longer a single known node, and the robot cannot tell which of the k possible worlds it is in.
- **Assumptions affected:** 'fully observable' (the localisation sensor is unreliable) and 'known start state' (now a set of k states). Determinism and a known map still hold.
- **Why knowledge becomes part of the state:** the effect of an action depends on the true location, which the robot does not know, and its next decision can depend only on what it knows. A plan must therefore work for every location it might be in, so the planning state has to be the set of locations still possible, called the **belief state**.

B. Evaluation of the original design (2 marks)

- **Can the original algorithm be applied?** Not directly. Running BFS/UCS from each of the k candidates gives k different routes, but the robot must execute one action sequence without knowing which candidate is real. The same algorithm can be applied if the 'state' becomes a belief state, a subset of V.
- **Size of the new space:** the belief space has up to $2^V - 1$ states. Since actions are deterministic, a belief never grows, so the beliefs reachable from a k-set have at most k members and number at most $\Sigma_{j=1..k} C(V, j)$. For the 12×12 floor plan with V = 124 open cells this bound is 7,750 for k = 2, 9,699,125 for k = 4 and 1.2×10^{12} for k = 8, compared with just 124 states in the original problem.
- **Is the original design acceptable?** For a single known start it is; for the belief problem it is not, because the state space explodes combinatorially with k and V (shown in section D), and a design that returns one route per candidate is not a valid plan.

C. Adaptation or redesign (2 marks)

Decision: MODIFY. The route-planning core (UCS on adjacency lists) is retained, and a belief-state layer is added in front of it.

- **Layer 1 – localise:** search over belief states. Result(b, a) = { move(s, a) : s in b }; goal: b = {dock} (all possible worlds at the dock) in the sensorless case. With a local sensor, the percept after each move splits the belief into the parts that give the same percept, and the plan becomes a conditional plan (an AND–OR search): one branch per percept, and the robot picks the branch matching what it senses.
- **Layer 2 – navigate:** as soon as the belief is a single junction, the robot knows where it is and runs the original UCS/BFS to the dock.
- **Effect on complexity:** sensorless search is a graph search over up to Σ C(V, j) belief states. With sensing, each belief of size m is split into at most p percept classes, so beliefs shrink quickly and far fewer are generated (measured below). The cost of the final navigation stage is unchanged, O((V + E) log V).

Model used in the experiment. A 12×12 floor plan (# = wall, dock at the top-left corner), with a compass so actions are N, S, E, W; moving into a wall leaves the robot where it is. The wall sensor reports which of the four neighbours are blocked.

```
.......##...
......#..#..
......#...#.
..#.#.......
..........#.
......#.....
..#..#......
...##......#
.#....##....
#...........
...#........
............
```

D. Evidence-based defence (2 marks)

Ten random initial beliefs per k (Random(29)); the sensorless search is a BFS over belief states capped at 150,000 generated beliefs; the sensor version is an AND–OR iterative-deepening search whose plan length is the worst-case depth of the conditional plan.

k	No sensor: solved	No sensor: beliefs generated (mean)	No sensor: plan length (moves)	Sensor: solved	Sensor: beliefs generated (mean)	Sensor: worst-case plan length
2	10/10	4,823	14.8 (max 22)	10/10	5.2	13.5 (max 18)
4	1/10	135,410 (cap 150,000)	9.0 (max 9) [1 trial]	10/10	14.4	15.6 (max 20)
8	0/10	150,001 (cap 150,000)	none found	10/10	52.8	18.3 (max 21)

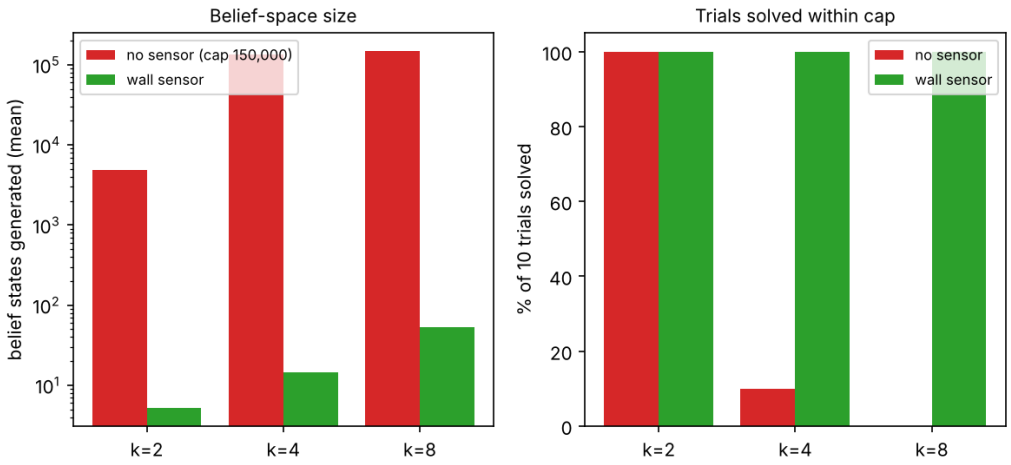


Figure 6 – Belief states generated (log scale) and trials solved, with and without the local sensor.

- **Without a sensor** the belief space blows up: at k = 2 about 4,823 beliefs were needed (all 10 trials solved, bound 7,750); at k = 4 only 1 of 10 trials finished inside the 150,000-belief cap and at k = 8 none did. The one k = 4 plan found (9 moves) comes from a lucky easy case and should not be compared with the others. The cap is a limit of my experiment, so I can only say these searches needed more than 150,000 beliefs, not how many they would need.
- **With the sensor** every trial was solved with 5.2, 14.4 and 52.8 belief states for k = 2, 4, 8, about 900 times fewer than the sensorless search at k = 2 and at least 2,800 times fewer at k = 8 (the sensorless search had already hit its cap). The worst-case plan length rose slowly with k (13.5, 15.6, 18.3 moves on average), because after a few moves the percepts isolate the true junction and the remaining moves are the ordinary shortest path (at most 22 moves from the farthest junction on this plan).
- **Conclusion:** the original single-state design is not sufficient; the belief-state formulation is necessary, and a cheap local sensor is what keeps it practical. The retained UCS handles the final navigation. If the sensor cannot be repaired, the system needs a much smaller floor-plan abstraction or additional landmarks to keep the belief space tractable.

Limits of this evidence: one 124-cell floor plan, 10 trials per k, an idealised compass and wall sensor, and caps on the sensorless search. The conclusions about direction and order of magnitude are supported; exact figures will differ on other floor plans.

Appendix – Source code

hospital_search.py

```
"""
CSA17 Assignment 1 - Hospital medicine-delivery robot
Uninformed search on generated hospital graphs.

Strategies
  1. BFS   - iterative, FIFO queue, explored set (fewest moves)
  2. UCS   - iterative, priority queue, explored set (least travel time)
  3. IDDFS - recursive depth-limited DFS repeated with growing limit (O(d) memory)
"""
import random
import sys
import time
from collections import deque
import heapq

sys.setrecursionlimit(20000)

SEED = 17          # master seed (reported in the document)
MAX_W = 10         # travel time of one move: integer minutes in [1, MAX_W]
EXTRA_EDGE_RATIO = 0.10   # extra cross-links (bridges, lifts) as fraction of V
SEALED_WING = 4    # nodes of a disconnected wing (used for the unreachable test)


# ---------------------------------------------------------------------------
# Graph generation + preprocessing
# ---------------------------------------------------------------------------
def generate_edge_list(n, seed=SEED):
    """Raw hospital map as an EDGE LIST (what a building-plan import gives us).
    Main part: random corridor tree rooted at node 0 (pharmacy) + 10% extra
    cross-links.  Last SEALED_WING nodes form a separate wing (unreachable)."""
    rng = random.Random(seed)
    main = n - SEALED_WING
    edges = []
    for v in range(1, main):
        u = rng.randrange(0, v)                  # attach to an earlier junction
        edges.append((u, v, rng.randint(1, MAX_W)))
    seen = {(min(u, v), max(u, v)) for u, v, _ in edges}
    for _ in range(int(EXTRA_EDGE_RATIO * main)):
        u, v = rng.randrange(main), rng.randrange(main)
        key = (min(u, v), max(u, v))
        if u != v and key not in seen:
            seen.add(key)
            edges.append((u, v, rng.randint(1, MAX_W)))
    for v in range(main + 1, n):                 # sealed wing: a small chain
        edges.append((v - 1, v, rng.randint(1, MAX_W)))
    return edges


def build_adjacency(n, edges):
    """PREPROCESSING: edge list -> adjacency lists. Theta(V+E) time and space."""
    nbrs = [[] for _ in range(n)]
    wts = [[] for _ in range(n)]
    for u, v, w in edges:
        nbrs[u].append(v); wts[u].append(w)
        nbrs[v].append(u); wts[v].append(w)
    return nbrs, wts


def path_cost(path, nbrs, wts):
    total = 0
    for a, b in zip(path, path[1:]):
        total += wts[a][nbrs[a].index(b)]
    return total


class Stats:
    __slots__ = ("generated", "expanded", "max_frontier", "iterations")
    def __init__(self):
```

```
        self.generated = 0; self.expanded = 0
        self.max_frontier = 0; self.iterations = 0


# ------------------------------------------------------------------------
# 1. BFS (iterative, queue + explored set, goal test at generation)
# ------------------------------------------------------------------------
def bfs(nbrs, start, goal):
    st = Stats()
    if start == goal:
        return [start], st
    parent = {start: None}                      # doubles as explored/reached set
    frontier = deque([start])
    st.max_frontier = 1
    while frontier:
        u = frontier.popleft()                  # <- node expansion
        st.expanded += 1
        for v in nbrs[u]:                       # <- successor generation
            if v in parent:                     # duplicate detection
                continue
            st.generated += 1
            parent[v] = u
            if v == goal:                       # early goal test
                return _rebuild(parent, v), st
            frontier.append(v)
        if len(frontier) > st.max_frontier:
            st.max_frontier = len(frontier)
    return None, st


# ------------------------------------------------------------------------
# 2. UCS (iterative, priority queue + explored set, goal test at expansion)
# ------------------------------------------------------------------------
def ucs(nbrs, wts, start, goal):
    st = Stats()
    best = {start: 0}
    parent = {start: None}
    frontier = [(0, start)]
    done = set()
    st.max_frontier = 1
    while frontier:
        g, u = heapq.heappop(frontier)          # <- node expansion (cheapest)
        if u in done:
            continue
        if u == goal:                           # goal test at expansion => optimal
            return _rebuild(parent, u), st
        done.add(u)
        st.expanded += 1
        nb, wt = nbrs[u], wts[u]
        for i in range(len(nb)):                # <- successor generation
            v = nb[i]
            ng = g + wt[i]
            if v not in done and ng < best.get(v, 1 << 60):
                best[v] = ng
                parent[v] = u
                st.generated += 1
                heapq.heappush(frontier, (ng, v))  # <- frontier insertion O(log)
        if len(frontier) > st.max_frontier:
            st.max_frontier = len(frontier)
    return None, st


# ------------------------------------------------------------------------
# 3. IDDFS (recursive depth-limited DFS, repeated for limit = 0,1,2,...)
#    No explored set: only the current path is checked (cycle avoidance).
# ------------------------------------------------------------------------
class BudgetExceeded(Exception):
    pass


def _dls(u, goal, limit, nbrs, path, on_path, st, budget):
    """Recursive depth-limited search. Returns 'found' | 'cutoff' | 'fail'."""
    if u == goal:                               # goal test
```

```
        return "found"
    if limit == 0:
        return "cutoff"
    st.expanded += 1
    if len(path) > st.max_frontier:
        st.max_frontier = len(path)
    cutoff = False
    for v in nbrs[u]:                           # <- successor generation
        if v in on_path:                        # cycle check (path only)
            continue
        st.generated += 1
        if st.generated > budget:
            raise BudgetExceeded
        path.append(v); on_path.add(v)
        res = _dls(v, goal, limit - 1, nbrs, path, on_path, st, budget)  # recursive call
        if res == "found":
            return "found"
        path.pop(); on_path.discard(v)
        if res == "cutoff":
            cutoff = True
    return "cutoff" if cutoff else "fail"


def iddfs(nbrs, start, goal, budget=5_000_000, max_depth=10_000):
    """Returns (path|None, stats, status) status in {'found','unreachable','budget'}"""
    st = Stats()
    try:
        for limit in range(max_depth + 1):
            st.iterations += 1
            path, on_path = [start], {start}
            res = _dls(start, goal, limit, nbrs, path, on_path, st, budget)
            if res == "found":
                return list(path), st, "found"
            if res == "fail":                   # whole tree exhausted, no cutoff
                return None, st, "unreachable"
    except BudgetExceeded:
        return None, st, "budget"
    return None, st, "budget"


def _rebuild(parent, v):
    out = []
    while v is not None:
        out.append(v); v = parent[v]
    return out[::-1]


# ---------------------------------------------------------------------------
# helpers used by the experiments
# ---------------------------------------------------------------------------
def bfs_depths(nbrs, start):
    d = {start: 0}
    q = deque([start])
    while q:
        u = q.popleft()
        for v in nbrs[u]:
            if v not in d:
                d[v] = d[u] + 1; q.append(v)
    return d


def timed(fn, *a, runs=5, **k):
    """Average wall-clock time over `runs` runs; returns (result, avg_seconds)."""
    times, res = [], None
    for _ in range(runs):
        t0 = time.perf_counter()
        res = fn(*a, **k)
        times.append(time.perf_counter() - t0)
    return res, sum(times) / len(times)
```

run_experiments.py

```
"""Tasks 8-9: correctness tests + scaling benchmark.  Usage: python run_experiments.py [n
...]"""
import json
import random
import sys
import time
from hospital_search import *

SIZES = [1000, 10000, 100000, 1000000]
IDDFS_BUDGET = 30_000_000          # max node generations before we declare 'budget'
RUNS = 5


def pick_goal(n, depths):
    """Reproducible far-ish ward: random node among those at >= 60% of max depth."""
    md = max(depths.values())
    cand = sorted(v for v in depths if depths[v] >= 0.6 * md)
    return random.Random(SEED + n).choice(cand)


def run_one(name, fn, runs):
    """Run once for stats, then time `runs` runs (fewer if the first run is slow)."""
    t0 = time.perf_counter(); res = fn(); first = time.perf_counter() - t0
    k = runs if first < 8 else (2 if first < 30 else 1)
    times = [first]
    for _ in range(k - 1):
        t0 = time.perf_counter(); fn(); times.append(time.perf_counter() - t0)
    return res, sum(times) / len(times), len(times)


def main():
    sizes = [int(a) for a in sys.argv[1:]] or SIZES
    out = []
    for n in sizes:
        t0 = time.perf_counter(); edges = generate_edge_list(n); t_gen = time.perf_counter()
- t0
        t0 = time.perf_counter(); nbrs, wts = build_adjacency(n, edges); t_prep =
time.perf_counter() - t0
        depths = bfs_depths(nbrs, 0)
        goal = pick_goal(n, depths)
        unreachable = n - 1                      # inside the sealed wing
        print(f"\n=== n={n}  E={len(edges)}  goal={goal} (depth {depths[goal]})  "
              f"prep={t_prep:.3f}s  seed={SEED}", flush=True)

        # ---------- correctness tests (all three algorithms) ----------
        tests = {}
        for label, g in (("reachable", goal), ("start==goal", 0), ("unreachable",
unreachable)):
            row = {}
            p, s = bfs(nbrs, 0, g); row["BFS"] = None if p is None else (len(p) - 1,
path_cost(p, nbrs, wts))
            p, s = ucs(nbrs, wts, 0, g); row["UCS"] = None if p is None else (len(p) - 1,
path_cost(p, nbrs, wts))
            p, s, st = iddfs(nbrs, 0, g, budget=IDDFS_BUDGET if label != "unreachable" else
2_000_000)
            row["IDDFS"] = st if p is None else (len(p) - 1, path_cost(p, nbrs, wts))
            tests[label] = row
            print(f"  test {label:12s} -> {row}", flush=True)

        # ---------- benchmark on the reachable pair ----------
        _, ucs_stats = ucs(nbrs, wts, 0, goal)
        pu, _ = ucs(nbrs, wts, 0, goal)
        opt_cost = path_cost(pu, nbrs, wts)
        rec = {"n": n, "E": len(edges), "goal": goal, "goal_depth": depths[goal],
               "max_depth": max(depths.values()), "t_gen": t_gen, "t_prep": t_prep, "tests":
tests,
               "opt_cost": opt_cost, "algos": {}}
        runners = {
            "BFS": lambda: bfs(nbrs, 0, goal),
            "UCS": lambda: ucs(nbrs, wts, 0, goal),
            "IDDFS": lambda: iddfs(nbrs, 0, goal, budget=IDDFS_BUDGET),
```

```
        }
        for name, fn in runners.items():
            res, avg_t, nruns = run_one(name, fn, RUNS)
            if name == "IDDFS":
                path, st, status = res
            else:
                path, st = res; status = "found"
            r = {"status": status, "generated": st.generated, "expanded": st.expanded,
                 "max_frontier": st.max_frontier, "time": avg_t, "runs": nruns}
            if path is not None:
                r["depth"] = len(path) - 1
                r["cost"] = path_cost(path, nbrs, wts)
                r["cost_optimal"] = r["cost"] == opt_cost
                r["iterations"] = st.iterations
            else:
                r["iterations"] = st.iterations
            rec["algos"][name] = r
            print(f"  {name:6s} {r}", flush=True)
        out.append(rec)
        json.dump(out, open(f"results_{'_'.join(map(str, sizes))}.json", "w"), indent=1)


if __name__ == "__main__":
    main()
```

exp_cases.py

```
"""Task 4 + 10 support: best / average / worst cases (n=10,000) and IDDFS per-iteration
growth."""
import json, random, statistics
from hospital_search import *
from hospital_search import _dls

n = 10000
edges = generate_edge_list(n); nbrs, wts = build_adjacency(n, edges)
dep = bfs_depths(nbrs, 0)
rng = random.Random(SEED + 99)
reach = sorted(v for v in dep if v != 0)
neighbor = nbrs[0][0]                       # goal = first successor of pharmacy
deepest = max(dep, key=lambda v: (dep[v], v))
avg_goals = rng.sample(reach, 30)
unreach = n - 1

def stats_for(goal, budget=20_000_000):
    p1, s1 = bfs(nbrs, 0, goal)
    p2, s2 = ucs(nbrs, wts, 0, goal)
    p3, s3, st = iddfs(nbrs, 0, goal, budget=budget)
    return {"BFS": s1.generated, "UCS": s2.generated, "IDDFS": s3.generated, "IDDFS_status":
st,
            "depth": dep.get(goal)}

res = {}
res["best_start_eq_goal"] = stats_for(0)
res["best_goal_is_neighbour"] = stats_for(neighbor)
av = [stats_for(g) for g in avg_goals]
res["average_30_random_goals"] = {
    "BFS": statistics.mean(a["BFS"] for a in av), "UCS": statistics.mean(a["UCS"] for a in
av),
    "IDDFS": statistics.mean(a["IDDFS"] for a in av), "mean_depth":
statistics.mean(a["depth"] for a in av),
    "IDDFS_statuses": sorted(set(a["IDDFS_status"] for a in av))}
res["worst_deepest_goal"] = stats_for(deepest)
res["worst_unreachable"] = stats_for(unreach, budget=3_000_000)
res["n"] = n; res["deepest_depth"] = dep[deepest]
res["component_size"] = len(dep)
res["reachable_edges_x2"] = sum(len(nbrs[v]) for v in dep)

# IDDFS: generated nodes for every depth limit towards `deepest` (n = 100,000, 1,000,000
omitted for time)
def per_limit(nb, goal, maxl):
    out = []
```

```
    for L in range(maxl + 1):
        st = Stats(); path, on = [0], {0}
        r = _dls(0, goal, L, nb, path, on, st, 10**9)
        out.append((L, st.generated, r))
    return out
n2 = 100000
e2 = generate_edge_list(n2); nb2, wt2 = build_adjacency(n2, e2)
d2 = bfs_depths(nb2, 0); md = max(d2.values())
g2 = max(d2, key=lambda v: (d2[v], v))
res["iddfs_levels_100k"] = per_limit(nb2, g2, d2[g2])
res["iddfs_levels_goal_depth"] = d2[g2]
# average out-degree (branching factor incl. parent) of the generated graphs
res["avg_degree"] = {n_: (2 * len(generate_edge_list(n_))) / n_ for n_ in (1000, 10000,
100000)}
json.dump(res, open("cases.json", "w"), indent=1)
print(json.dumps(res, indent=1))
```

belief_search.py

```
"""Task 12: localisation failure -> search over BELIEF STATES (sensorless) and with a local
wall sensor.

Floor plan: W x H grid of corridor cells, some blocked (walls / rooms).  The robot has a
compass, so actions are
N,S,E,W.  Moving into a wall leaves the robot where it is (bump).  After a restart it only
knows it is in one of k
open cells (its belief state).  Goal: belief state == {dock}.
"""
import random, sys, json, statistics, time
from collections import deque
from hospital_search import SEED

W = H = 12
WALL_DENSITY = 0.18
ACTIONS = ("N", "S", "E", "W")
DELTA = {"N": (-1, 0), "S": (1, 0), "E": (0, 1), "W": (0, -1)}


def make_floor(seed=SEED):
    rng = random.Random(seed)
    while True:
        grid = [[rng.random() < WALL_DENSITY for _ in range(W)] for _ in range(H)]  # True =
wall
        open_cells = [(r, c) for r in range(H) for c in range(W) if not grid[r][c]]
        seen = {open_cells[0]}; q = deque([open_cells[0]])
        while q:
            r, c = q.popleft()
            for dr, dc in DELTA.values():
                nr, nc = r + dr, c + dc
                if 0 <= nr < H and 0 <= nc < W and not grid[nr][nc] and (nr, nc) not in seen:
                    seen.add((nr, nc)); q.append((nr, nc))
        if len(seen) == len(open_cells) and not grid[0][0]:   # connected floor, dock cell
(0,0) free
            return grid, open_cells


GRID, CELLS = make_floor()
IDX = {cell: i for i, cell in enumerate(CELLS)}
DOCK = IDX[(0, 0)]                                  # dock station in the north-west corner


def step(i, a):
    r, c = CELLS[i]; dr, dc = DELTA[a]; nr, nc = r + dr, c + dc
    if 0 <= nr < H and 0 <= nc < W and not GRID[nr][nc]:
        return IDX[(nr, nc)]
    return i                                        # bump: stay


MOVE = [[step(i, a) for a in ACTIONS] for i in range(len(CELLS))]
```

```
PERCEPT = [tuple(MOVE[i][j] == i for j in range(4)) for i in range(len(CELLS))]   # which of
N,S,E,W are blocked


def dist_to_dock():
    d = {DOCK: 0}; q = deque([DOCK])
    while q:
        u = q.popleft()
        for j in range(4):
            v = MOVE[u][j]
            if v not in d:
                d[v] = d[u] + 1; q.append(v)
    return d
DIST = dist_to_dock()


# ---------------- (a) sensorless: BFS over belief states -----------------
def sensorless(belief0, cap=150_000):
    start = frozenset(belief0)
    goal = frozenset([DOCK])
    if start == goal:
        return 0, 1, "solved"
    seen = {start: None}; q = deque([start]); generated = 1
    while q:
        b = q.popleft()
        for j in range(4):
            nb = frozenset(MOVE[s][j] for s in b)          # deterministic: |nb| <= |b|
            if nb in seen:
                continue
            seen[nb] = (b, j); generated += 1
            if nb == goal:
                n = 0; cur = nb
                while seen[cur] is not None:
                    cur = seen[cur][0]; n += 1
                return n, generated, "solved"
            if generated > cap:
                return None, generated, "cap"
            q.append(nb)
    return None, generated, "no plan"                      # whole reachable belief space
exhausted


# ---------------- (b) with a local wall sensor: AND-OR minimax ----------------
def split(b):
    parts = {}
    for s in b:
        parts.setdefault(PERCEPT[s], []).append(s)
    return [frozenset(v) for v in parts.values()]


CALL_CAP = 400_000


def with_sensor(belief0, hmax=60):
    """Worst-case length of a conditional plan: iterative deepening on the plan depth h.
    Percepts after every move split the belief; once the belief is a singleton the robot
navigates
    along the shortest path (this is the original BFS design reused)."""
    seen_beliefs = set()
    memo = {}
    calls = [0]

    def can(b, h):
        calls[0] += 1
        if calls[0] > CALL_CAP:
            raise TimeoutError
        seen_beliefs.add(b)
        if len(b) == 1:
            return DIST[next(iter(b))] <= h
        if h == 0:
            return False
        key = (b, h)
```

```
        if key in memo:
            return memo[key]
        ok = False
        for j in range(4):
            nb = frozenset(MOVE[s][j] for s in b)
            if nb == b:
                continue                                        # useless action
            if all(can(p, h - 1) for p in split(nb)):
                ok = True; break
        memo[key] = ok
        return ok

    parts = split(frozenset(belief0))
    plan_len = 0
    try:
        for p in parts:
            h = 0
            while not can(p, h):
                h += 1
                if h > hmax:
                    return None, len(seen_beliefs) + len(parts), "no plan"
            plan_len = max(plan_len, h)
    except TimeoutError:
        return None, len(seen_beliefs) + len(parts), "cap"
    return plan_len, len(seen_beliefs) + len(parts), "solved"


def main(trials=10, ks=(2, 4, 8)):
    rng = random.Random(SEED + 12)
    out = {"cells": len(CELLS), "dock": CELLS[DOCK], "grid": ["".join("#" if x else "." for x
in row) for row in GRID],
           "rows": []}
    print("floor plan ('#'=wall, dock at top-left):")
    print("\n".join(out["grid"]))
    print(f"open cells V={len(CELLS)}  max shortest-path distance to
dock={max(DIST.values())}")
    for k in ks:
        sl, sg, ss, wl, wg, ws = [], [], 0, [], [], 0
        nst, wst = [], []
        t0 = time.perf_counter()
        for t in range(trials):
            b0 = rng.sample(range(len(CELLS)), k)
            n, g, st = sensorless(b0); sg.append(g); nst.append(st)
            if st == "solved": sl.append(n); ss += 1
            n, g, st = with_sensor(b0); wg.append(g); wst.append(st)
            if st == "solved": wl.append(n); ws += 1
        row = {"k": k, "trials": trials,
               "nosensor_solved": ss, "nosensor_plan_mean": statistics.mean(sl) if sl else
None,
               "nosensor_plan_max": max(sl) if sl else None, "nosensor_beliefs_mean":
statistics.mean(sg),
               "sensor_solved": ws, "sensor_plan_mean": statistics.mean(wl) if wl else None,
               "sensor_plan_max": max(wl) if wl else None, "sensor_beliefs_mean":
statistics.mean(wg),
               "nosensor_status": {x: nst.count(x) for x in set(nst)}, "sensor_status": {x:
wst.count(x) for x in set(wst)},
               "seconds": time.perf_counter() - t0}
        out["rows"].append(row); print(row, flush=True)
    # naive original design: plan from every candidate junction separately would need k
plans; baseline
    out["theory_bound_k"] = {k: sum(__import__("math").comb(len(CELLS), j) for j in range(1,
k + 1)) for k in (2, 4, 8)}
    json.dump(out, open("belief_results_%s.json" % "_".join(map(str, ks)), "w"), indent=1)


if __name__ == "__main__":
    ks = tuple(int(a) for a in sys.argv[1:]) or (2, 4, 8)
    main(ks=ks)
```

make_figs.py

```
import json, glob
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
R = json.load(open("results_1000_10000_100000.json")) +
json.load(open("results_1000000.json"))
C = json.load(open("cases.json"))
ns = [r["n"] for r in R]
col = {"BFS": "#1f77b4", "UCS": "#d62728", "IDDFS": "#2ca02c"}
def series(a, key): return [r["algos"][a][key] for r in R]
def save(name): plt.tight_layout(); plt.savefig(f"fig/{name}.png", dpi=150); plt.close()

for key, ttl, yl, name in [("generated","Nodes generated vs. state-space size","nodes
generated","f1_generated"),
                           ("max_frontier","Maximum frontier / stack size (memory
proxy)","max frontier (nodes)","f2_frontier"),
                           ("time","Execution time (mean of runs) vs. state-space
size","seconds","f3_time")]:
    plt.figure(figsize=(6.2,4))
    for a in col: plt.plot(ns, series(a,key), "o-", label=a, color=col[a])
    if key != "max_frontier":
        plt.plot(ns, [series("BFS",key)[0]*n/ns[0] for n in ns], "k:", label="linear (Θ(V))
reference")
    plt.xscale("log"); plt.yscale("log"); plt.xlabel("states V"); plt.ylabel(yl);
plt.title(ttl, fontsize=10)
    plt.grid(True, which="both", alpha=.3); plt.legend(fontsize=8); save(name)

lv = C["iddfs_levels_100k"]
plt.figure(figsize=(6.2,4))
L=[x[0] for x in lv[1:]]; G=[x[1] for x in lv[1:]]
plt.semilogy(L, G, "o-", color=col["IDDFS"], label="IDDFS nodes generated per depth limit
(V=100,000)")
plt.axhline(100000, color="k", ls="--", label="V = 100,000 states")
plt.xlabel("depth limit ℓ"); plt.ylabel("nodes generated in one DLS pass"); plt.grid(True,
which="both", alpha=.3)
plt.legend(fontsize=8); plt.title("Iterative deepening: exponential growth ≈ ×2.05 per
level", fontsize=10); save("f4_iddfs_levels")

import numpy as np
cases = ["start=goal","goal = 1st\nneighbour","average\n(30 goals)","deepest goal\n(depth
15)","unreachable"]
vals = {a: [C["best_start_eq_goal"][a], C["best_goal_is_neighbour"][a],
C["average_30_random_goals"][a],
            C["worst_deepest_goal"][a], C["worst_unreachable"][a]] for a in col}
x = np.arange(5); w=.27
plt.figure(figsize=(7,4))
for i,a in enumerate(col): plt.bar(x+(i-1)*w, [max(v,0.5) for v in vals[a]], w, label=a,
color=col[a])
plt.yscale("log"); plt.xticks(x, cases, fontsize=8); plt.ylabel("nodes generated (log)");
plt.legend(fontsize=8)
plt.title("Best / average / worst case, V = 10,000 (IDDFS unreachable: stopped at 3M
budget)", fontsize=9); plt.grid(True, axis="y", alpha=.3); save("f5_cases")

B = {}
for k in (2,4,8): B[k]=json.load(open(f"belief_results_{k}.json"))["rows"][0]
ks=[2,4,8]; fig,ax=plt.subplots(1,2,figsize=(8,3.8))
ax[0].bar(np.arange(3)-.2,[B[k]["nosensor_beliefs_mean"] for k in ks],.4,label="no sensor
(cap 150,000)",color="#d62728")
ax[0].bar(np.arange(3)+.2,[B[k]["sensor_beliefs_mean"] for k in ks],.4,label="wall
sensor",color="#2ca02c")
ax[0].set_yscale("log"); ax[0].set_xticks(range(3)); ax[0].set_xticklabels([f"k={k}" for k in
ks]); ax[0].set_ylabel("belief states generated (mean)")
ax[0].legend(fontsize=8); ax[0].set_title("Belief-space size",fontsize=10)
ax[1].bar(np.arange(3)-.2,[B[k]["nosensor_solved"]*10 for k in
ks],.4,color="#d62728",label="no sensor")
ax[1].bar(np.arange(3)+.2,[B[k]["sensor_solved"]*10 for k in
ks],.4,color="#2ca02c",label="wall sensor")
ax[1].set_xticks(range(3)); ax[1].set_xticklabels([f"k={k}" for k in ks]);
ax[1].set_ylabel("% of 10 trials solved"); ax[1].legend(fontsize=8)
ax[1].set_title("Trials solved within cap",fontsize=10); save("f6_belief")
print("ok")
```